# Programming 3

Homework Specification

## 1 Introduction

This document outlines the main goals and the overall expected functionality of the program that will be developed as a homework project for the subject **Programming 3** at the **Budapest University of Technology and Economics**. The aim of the project is to apply the concepts and techniques learned during the semester to design and implement a complete software application. The document serves as a specification that defines the scope of the project, described its planned functionality a high level, and provides an overview of the methods and tools that will be used during development.

## 2 Description of the Task

The program to be developer is a **Text Editor Application** that implements basic word-processing functionality within a simple graphic user interface. The goal of the application is to allow users to create, edit, and save text documents in a simple and user-friendly graphical interface. The project aims to to demonstrate the use of OOP principles, event-driven GUI design, and file input/output operations in practice.

The editor will include standard text formatting features such as **bold**, **italic**, **underline**, **font size adjustment**, **font selection**, and **text alignment**. Users will be able to perform common file operations such as **creating new documents**, **opening existing files**, and **saving their work**. The program will also include a **menu bar** and **toolbar**, and as well as a status bar to display document information such as word or character count.

Additionally, the editor will provide a **Find tool** that allows users to search for specific words or phrases within the document. Matching text will be **temporarily highlighted**, making it easier to users to locate occurrences of the searched term without altering the content.

As a reference, it is planned to use **Microsoft Word** and/or **LibreOffice** to get ideas about layout, menu organization, and basic formatting options. It is not intended to replicate them, but rather get inspiration to implement a simpler version of those.

From a technical viewpoint, the program will be implemented using the **Swing** framework for the Graphical User Interface (GUI), **Java Serialization** for saving and loading files, and the **Collections Framework** to manage recent files and other internal data structures. Additionally, **JUnit Tests** will be written to verify different components of the program.

# 3 Use-Case Diagrams and Descriptions

The following figure illustrates the main interactions between the user and the text editor application. Each use case represents a core functionality of the system that the user can perform through the graphical interface.

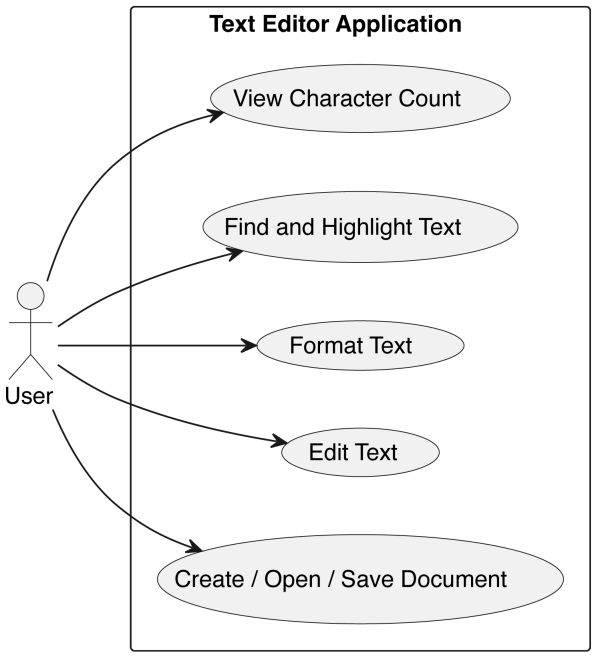


Figure 1: Use case diagram of the project

1. **View Character Count**
   - **Actor**: User
   - **Goal**: Count number of characters and words.
   - **Preconditions**: A document must be open and contain text.
   - **Basic Flow**: The system automatically updates the current number of characters and words in the document.
   - **Postconditions**: Status bar displays the current character count for the open document.
2. **Find and Highlight Text**
   - **Actor**: User
   - **Goal**: Locate all occurrences of a specific word or phrase in the document.
   - **Preconditions**: A document must be open and contain text
   - **Basic Flow**: The user selects "Find" from the menu. A dialog appears where the user enters a search item. The system scans the document and highlights all matches. The user can clear the highlights or repeat the search as needed.
   - **Postconditions**: All occurrences remain highlighted until the user cancels or performs a new search.
3. **Format Text**
   - **Actor**: User
   - **Goal**: Change the appearance of selected text
   - **Preconditions**: A document must be open

- **Basic Flow**: The user selects a portion of text and applies one of more formatting options, such as bold, italic, underline, text alignment, or font and size changes. The system updates the visual style of the selected text to reflect the chosen formatting
- **Postconditions**: The selected text is displayed with the applied formatting.

4. **Edit Text**
   - **Actor**: User
   - **Goal**: Modify the content of the document
   - **Preconditions**: A document is open in the editor
   - **Basic Flow**: The user types, deletes, or replaces text using the keyboard. All changes are instantly visible in the editor area.
   - **Postconditions**: The document reflects the performed changes.
5. **Create/Open/Save Document**
   - **Actor**: User
   - **Goal**: Manage text documents by creating new files, opening existing ones, or saving work.
   - **Preconditions**: The application is running.
   - **Basic Flow**: The user selects an option from the "File" menu or toolbar. Choosing "New" opens a new empty document, "Open" displays a file chooser dialog to load an existing file, and "Save" writes the current content to disk
   - **Postconditions**: The document is successfully created, opened, or saved, depending on the chosen action.

# 4 How the Task Will Be Solved

The program will be developed in Java using Swing to implement the graphical user interface. The initial idea is to use components such as `JTextPane` or `JEditorPane` for text editing and formatting, as they provide built-in support for styled text. Other elements such as viewports and scroll panes may also be used to manage document navigation. A menu bar will be placed at the top of the window to access the different operations, and a status bar at the bottom will display information such as character count. For text searching, a trivial implementation will be considered, where the program locates all occurrences within the text and then highlights it. File operations (create, open, save) will be handled using Java's standard file input/output and serialization mechanisms. Afterwards, some Unit Tests will be included to test the features.

# 5 Nice-to-Have Features

In addition to the previously defined functionalities, several optional enhancements may be implemented to add capabilities to the text editor. **Those features are not essential for the base version**, but will be considered if time permits to take the challenge of developing it.

One common feature of all text editors is the possibility of exporting documents in formats like **PDF** or **DOCX**. Therefore, the addition of **document export options**, such as **PDF** generation using the **iText library** (or any other library) and an experimental attempt to export the document as a **.docx** file. This would make the editor more practical.